Agentic AI Workload Characterization

Yichao Yuan*, Ankita Nayak†, Souvik Kundu‡, Nishil Talati*
* *University of Illinois Urbana-Champaign, USA*
{yichaoy2,nishil}@illinois.edu
† *Gimlet Labs*, USA ‡ *Intel Corporation*, USA
ankitanayak@gimletlabs.ai souvikk.kundu@intel.com

arXiv:2605.26297v2 [cs.DC] 21 Sep 2026

***Abstract*—Agentic AI shifts LLM serving from isolated prompt-generation requests to stateful, multi-turn executions that repeatedly invoke the model, call tools, and grow context over time. This paper characterizes ReAct-style agents from both the LLM-serving and tool-execution perspectives using an end-to-end tracing infrastructure across reasoning and non-reasoning Gemma and Qwen configurations on five agentic benchmarks. Our study shows that agentic workloads are not simply long-prompt workloads: with effective context caching, most input tokens are reused across turns, making execution decode-dominated while increasing dependence on long-lived KV-cache state. We also find that tool use has a clear temporal structure, with agents shifting from read/explore behavior early in execution to execute/write behavior later. These results show that efficient agentic serving must jointly manage repeated model re-entry, persistent context state, and workload-dependent tool behavior.**

***Index Terms*—Agentic AI, large language models, workload characterization, LLM serving, KV cache, tool use**

I. Introduction

Large Language Models (LLMs) have accelerated the broader impact of AI, transforming several domains, including scientific discovery and software engineering, from AlphaFold-style protein-structure prediction [1] to AI-assisted programming with tools such as GitHub Copilot [2]. The next frontier is **Agentic AI** [3]–[5]: systems that combine LLMs with tools, execution environments, memory, and control logic to pursue goals over multiple steps. Unlike a single LLM call, an agent repeatedly reasons, acts, observes outcomes, and adapts [3], making it attractive for software engineering [6], data analytics [7], general-assistant workflows [8], and web-based tasks [9].

Agentic systems span ReAct-style reason-act-observe loops [3], Reflexion-style self-reflection [10], Toolformer-style tool learning [11], and LATS-style planning/search [12], differing in how they organize reasoning, tool use, observations, planning, and delegation. We focus on **ReAct-style agents** because they are simple, widely used, and closely match the serving loop of systems such as Claude Code [13], Codex [14], and OpenClaw [15]. Prior work studies agent capability, progress, behavior, cost, and CPU-centric execution [4], [5], [16], [17]. However, they leave open a systems-level question: *how do model serving, context growth, tool latency, and failure-driven retries interact to determine the efficiency bottlenecks of end-to-end agent execution?*

To address this gap, this paper characterizes ReAct-style agent workloads from both the LLM-serving and tool-execution perspectives. In specific, we develop a tracing infrastructure that runs Claude Code agents through Harbor [18], serves Gemma [19] and Qwen [20] models with vLLM [21], and correlates agent trajectories with request-level serving traces. Our benchmark suite spans data analytics, general assistant tasks, software engineering, and terminal environments using ADE-Bench [7], DABstep [22], GAIA [8], SWE-bench Pro [6], and Terminal-Bench 2.0 [23].

Our characterization shows that agentic workloads are long-tailed in two distinct ways: some tasks require many turns, while others accumulate large contexts with fewer turns. Surprisingly, *reasoning does not always increase cost*; for example, for Gemma, thinking reduces pathological trajectories, with ADE averaging 18.0 turns for Gemma Thinking versus 108.8 turns and a 786-turn maximum for Gemma Instant. Meanwhile, *context growth is not simply a proxy for turn count*: SWE-bench Pro has the largest accumulated contexts, averaging 69K–80K tokens, despite other workloads having more extreme turn-count tails. Thus, agentic serving must manage both repeated model re-entry and sustained KV-cache growth.

Our performance characterization shows that *agentic LLM serving is not conventional long-prompt serving with an effective context cache design.* Most input tokens are reused across turns, yielding high empirical cache-hit ratios of 84.6–99.5% and making decode dominate 91.0–98.6% of LLM time. Thus, efficient agent serving depends less on optimizing isolated long prompts and more on preserving long-lived KV cache state across turns; if this state is evicted, a decode-dominated workload can become expensive recomputation. While our analysis exposes the *potential of high cache reuse* in agentic workloads, achieving such decode-dominated execution in practice can be challenging because concurrent agents with long multi-turn contexts can exceed GPU memory, requiring cache-retention, scheduling, offloading, or recomputation mechanisms that introduce additional overheads [24]–[27].

Our tool characterization shows that tool execution is not a single workload category: tool calls range from lightweight file inspection to shell execution, web retrieval, delegated subagent work, and large returned observations. GAIA, for example, spends up to 28.7% of time in tools due to WebFetch/WebSearch, while SWE-bench Pro and Terminal-Bench rely heavily on Bash, Read, Edit, Agent, and TaskOutput. *Latency-heavy tools are not always failure-prone*: Agent and TaskOutput can dominate time and tokens, whereas Bash, Edit, and Read more often trigger retries, error observations,

TABLE I
COMPARISON WITH CLOSELY RELATED AGENT CHARACTERIZATION STUDIES.

Category / Characterization	Kim et al. [16]	Raj et al. [17]	Ma et al. [5]	Ours
Tool behavior *Latency, result size, failure behavior, and intention evolution*				✓
Agent execution behavior *Reasoning vs. non-reasoning model behavior*				✓
Agent execution behavior *Turn-count distribution and context growth across turns*	Both	Turn count only	Context growth only	Both
Runtime performance *LLM vs. tool execution-time breakdown*	✓	✓		✓

and context growth. In some cases, repeated tool failures can trap an agent in a recovery loop, causing long executions that consume turns and context without making meaningful progress. We also analyze how tool interactions evolve over a task, finding that *agents shift from early read/explore behavior for gathering information to later execute/write behavior for editing, testing, commands, and finalization.*

This work provides a unified, end-to-end characterization of agent behavior, context-state evolution, and serving-time performance using one tracing infrastructure built around Claude Code, a production-grade agent harness. Prior work studies only subsets of these dimensions and does not characterize them jointly. We also identify runtime-observable signals with direct performance implications, providing a practical basis for system optimization beyond behavioral observations. We summarize key differences in Table I. In summary, this paper makes the following contributions:

- We develop an end-to-end tracing infrastructure to characterize agentic AI workloads across model serving and tool execution.
- We characterize execution behavior, context growth, token reuse, and KV-cache dynamics across representative agent workloads.
- We analyze the cost, latency, and failure behavior of tool interactions and their impact on end-to-end agent execution.

II. Background

A. LLM Agents and the ReAct Paradigm

LLM agents are systems that use LLMs to iteratively reason, plan, and interact with external tools or environments to accomplish user-specified goals [3], [10]–[12]. They shift the LLM serving paradigm from isolated single-turn prompt–generation requests to a closed-loop, multi-turn execution process, in which agents repeatedly interact with the model, external tools, and the environment over an evolving context [28]–[31]. Agents can follow different organizational patterns, which vary in how they orchestrate LLM invocations, incorporate environmental observations, and determine subsequent actions; representative examples include ReAct [3], Reflexion [10], Toolformer-style tool use [11], and LATS [12]. Among these, ReAct is one of the most widely adopted paradigms due to its effectiveness and simplicity [3]. ReAct executes agents as a reason-act-observe loop: the LLM reasons about the next step, invokes tools or the environment, and incorporates the observation into later turns [3]. This makes agent execution a multi-turn process over an evolving context [24], [31]. It underlies many contemporary CLI agents, including Claude Code [13] and Codex [14], and is the focus of this paper.

B. Reasoning LLMs and Agents

Reasoning LLMs are models trained to generate additional intermediate tokens for deliberation before producing a final response, which can improve their ability to solve complex tasks and increase task success rates [3], [10], [12]. This capability is now supported across a broad range of state-of-the-art models, from large proprietary systems such as GPT-5.5 [32] to small- and mid-sized open-weight models such as Gemma 4 [19] and Qwen3.6 [20]. Motivated by these gains, modern LLM agents increasingly incorporate reasoning LLMs to improve their effectiveness on complex, multi-step tasks [10], [12], [33]. When used in ReAct agents, reasoning LLMs generate such intermediate tokens at every agent step before issuing tool calls or taking other actions [3]. These tokens are retained in the evolving context until the current user request is completed, thereby increasing both generation length and the agent's context usage, i.e., KV cache of conversation history [21], [24], [26].

Figure 1 illustrates a ReAct-style agent execution and the corresponding evolution of the serving context. At each agent step, the LLM consumes the accumulated context, generates intermediate reasoning tokens, a message, and optionally one or more tool calls; the environment then executes these calls and appends the returned observations to the next LLM input [3], [11]. A single LLM response may issue multiple tool calls before control returns to the model, causing multiple tool-use records and observations to be added within one step [11]. As the agent progresses, most previous input tokens can be reused through the prefix/KV cache [34], while newly generated reasoning tokens, messages, tool calls, and tool results extend the context and increase the amount of state that must be maintained for subsequent steps [21], [24], [26].

C. Agentic Serving Terminologies

Formally, we study the serving of ReAct-style agents powered by both reasoning and non-reasoning modes of LLMs. We describe a ReAct-style agent as a multi-step interaction process [3]. For an agent a, let $H_{a,i}$ denote its accumulated context (trajectory) before the i-th LLM invocation, and let

$$C_{a,i} = |H_{a,i}| \tag{1}$$

be the corresponding context length.

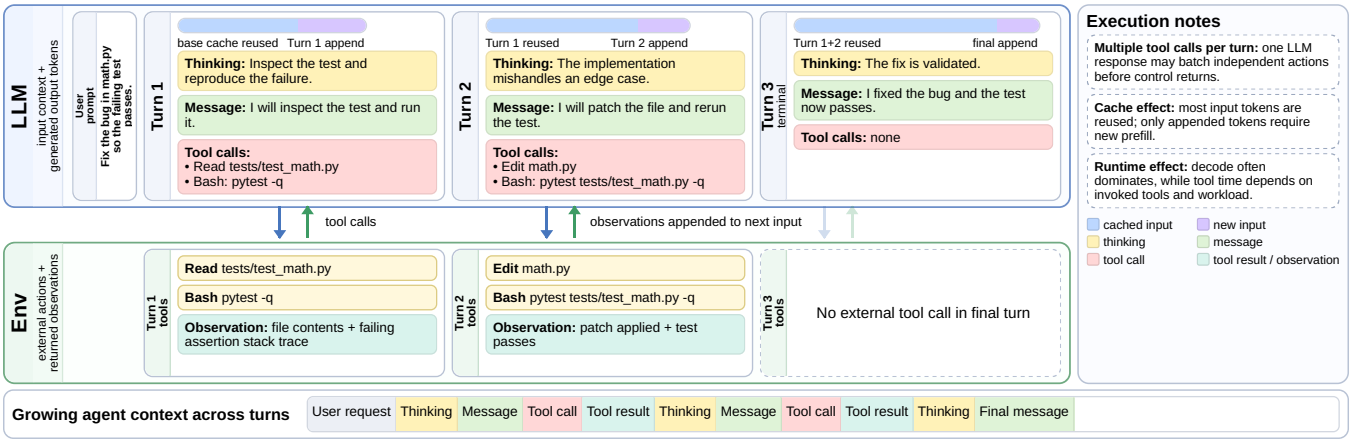

Fig. 1. ReAct agent execution and context growth example. Each turn consists of an LLM response, one or more tool calls, and environment observations. Across turns, prior reasoning, messages, tool calls, and observations are appended to the agent context. If a tool call launches a subagent, we treat the subagent execution as part of the parent agent's tool call.

At step i, conditioned on $H_{a,i}$, the agent uses the *LLM serving system* to generate an assistant-side response with a model that may contain three components:

$$z_{a,i} = (\theta_{a,i},\ m_{a,i},\ u_{a,i}), \tag{2}$$

where $\theta_{a,i}$ denotes intermediate reasoning, or *thinking*, tokens, $m_{a,i}$ denotes *message* tokens, and $u_{a,i}$ denotes *tool-call* tokens. Some components may be empty, and their exact ordering is model- and chat-template-dependent. For non-reasoning models (no thinking), $\theta_{a,i} = \varnothing$.

Before the next LLM invocation, these generated components are incorporated into the agent history according to the model's chat template. If tool calls are issued, the *environment* executes them and appends the tool-use results $o_{a,i}$ to the history. The context therefore evolves as

$$H_{a,i+1} = H_{a,i} \| \Phi(\theta_{a,i},\ m_{a,i},\ u_{a,i}) \| o_{a,i}, \tag{3}$$

where $\Phi(\cdot)$ denotes chat-template formatting and $\|$ denotes concatenation. For a terminal step that completes normally, no tool calls or tool-use results are produced, i.e., $u_{a,i} = o_{a,i} = \varnothing$. We define an *agent turn* as one interaction in which the agent issues tool call(s) and receives their results.

Following a common pattern in contemporary CLI agents such as Claude Code [35], an agent may invoke tools that launch additional agents. We refer to the initiating agent as the *main agent* and each launched agent as a *subagent*. Consistent with Claude Code's convention, we do not allow nested subagent invocations in this work [35].

III. Characterization Methodology

This work aims to comprehensively characterize the performance characteristics and behavioral patterns of real-world LLM agents across different tasks. The design space for agentic workloads is broad, spanning many agent frameworks, models, benchmarks, tools, and task domains, and an exhaustive study of this space is beyond the scope of a single paper. We therefore focus on a representative slice of this space, as detailed below. To this end, we evaluate agents under diverse model configurations, including reasoning and non-reasoning modes, and across multiple agentic benchmarks. We use Claude Code [13] as the agent framework throughout our study. Concretely, our characterization spans the following dimensions:

- **Models.** We evaluate two open-weight LLMs: Qwen3.6-27B [20] and Gemma4-31B [19].
- **Reasoning configurations.** For each model, we evaluate both thinking and non-thinking modes. We denote the non-thinking mode as the model's *Instant* variant, while using the raw model name for its thinking mode. The two modes are enabled by setting the corresponding knob in the model's chat template.
- **Benchmark datasets.** We evaluate agents across a diverse set of agentic workloads, including:
  - **ADE-Bench** [7]: data analytics workloads,
  - **DABStep*** [22], data agent tasks,
  - **GAIA** [8], representing general-purpose assistant tasks that require multi-step reasoning and tool use,
  - **SWE-bench Pro*** [6], software engineering tasks,
  - **Terminal-Bench 2.0** [23]: terminal-based software engineering and system interaction tasks.

*For DABStep and SWE-bench Pro, the full benchmark suites are large and expensive to run end-to-end with multiple models and reasoning configurations. We therefore randomly sample 100 tasks from each benchmark suite to keep the study computationally tractable while still preserving task diversity within each workload. This allows us to collect fine-grained per-turn, per-request, and per-tool traces across all configurations, rather than limiting the characterization to a smaller set of models or metrics.

A. Characterization Infrastructure

We build an end-to-end characterization infrastructure to execute agents, serve models, and collect fine-grained

request-level traces. All experiments are conducted on a server equipped with two NVIDIA H100 NVL GPUs connected by 12 NVLink links, and an Intel Xeon Platinum 8592+ CPU. We serve all models using vLLM v0.20.0 [21] from the official Docker image, with tensor parallelism set to 2. We enable vLLM's OpenTelemetry support [36] and collect trace data through Jaeger [37], which provides detailed timing information for each request, including prefill and decode execution time.

We use Harbor's evaluation environment [18], which setup a single agent execution's objective and execution environment, to launch and evaluate agent tasks. We use Claude Code [13] as the agent scaffold. To associate low-level LLM serving traces with high-level agent execution, we implement a customized request gateway between the agent and the vLLM server. The gateway has two components.

- **Forwarding proxy.** The proxy forwards every LLM request issued by the agent to the vLLM server while capturing and logging the request metadata. It also modifies each request to ensure that vLLM emits OpenTelemetry traces [36] for that request, allowing us to correlate agent-level interactions with serving-level metrics such as prefill time, decode time, and request latency.
- **Per-agent wrapper.** The wrapper launches each agent process and redirects its LLM traffic to the gateway. For each agent execution process, the wrapper assigns a unique API key, which allows the gateway to group all requests belonging to the same agent execution. This enables per-agent reconstruction of request sequences and supports subsequent analysis of both system performance and agent behavior.

Figure 2 summarizes our characterization infrastructure. Harbor launches isolated Claude Code agent executions, whose LLM requests are redirected through a gateway that records agent-level metadata and forwards requests to vLLM. vLLM emits OpenTelemetry traces collected by Jaeger, and the characterization pipeline combines these serving traces with agent request logs, action timing, and execution results.

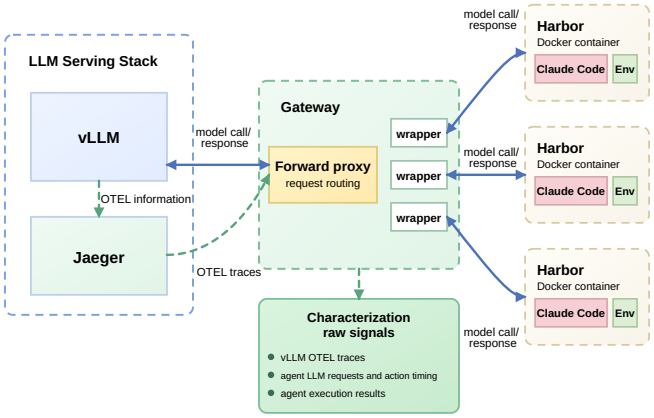

Fig. 2. Tracing infrastructure for characterizing agent workload components and their interactions. Harbor launches Dockerized Claude Code agents, whose model traffic is redirected by a wrapper through our gateway. The gateway forwards requests to vLLM, records agent-side request and action timing, and attaches metadata for per-agent trace correlation. vLLM emits OpenTelemetry (OTEL) traces collected by Jaeger; these traces, together with gateway logs and Harbor execution results, form the raw signals used in our characterization.

IV. Agent Execution Characterization

This section characterizes how agentic workloads evolve over the task lifetime. We ask four related questions: (1) how many turns agents execute, (2) how much context they accumulate across those turns, (3) what is the composition of generated output tokens, and (4) whether successful and failed runs impose different context demands.

Agent Turn Counts. Figure 3 shows that agent turn count is highly dependent on both workload and model behavior. Although most configurations average between 12–62 turns per task, several exhibit extreme long tails. Gemma Instant is the clearest example, reaching 786 turns on ADE, 640 on DABStep, 617 on GAIA, and 470 on Terminal Bench. In contrast, Gemma Thinking remains much more compact on most datasets, suggesting that explicit reasoning can reduce inefficient multi-turn exploration.

The benefit of reasoning is strongest for Gemma. On ADE, Gemma Thinking averages only 18.0±15.3 turns, while Gemma Instant averages 108.8±178.7 turns. Qwen shows a milder but still visible gap, with Qwen Instant usually requiring more turns than Qwen Thinking. SWE-bench Pro is a notable case: Qwen Instant averages 62.4 turns and shows a concentrated high-turn distribution, indicating consistently long trajectories rather than a few outliers. These results show that agentic execution lifetimes are both long-tailed and model-dependent, which has direct implications for serving systems because each additional turn requires re-entering the model, extending context, maintaining KV cache state, and scheduling another prefill/decode phase.

The extreme tail for ADE is driven primarily by the Gemma Instant configuration, where Table IV shows an unusually high number of failing 'Edit' calls: 2,757 edit attempts with a 95.4% failure rate. This suggests that *the agent is not merely taking many productive steps, but is trapped in a repeated edit-failure pattern in which it attempts to modify files, receives failure/error observations, and then retries without successfully recovering.* This behavior is consistent with Figure 3, where Gemma Instant on ADE reaches 786 turns and averages 108.8 turns, far above the corresponding Gemma Thinking configuration. Importantly, this pathological failure loop is not observed at the same severity in the other model–dataset combinations, indicating that the long tail is caused by a specific interaction between the non-thinking Gemma model and ADE's edit-heavy task structure. Thus, *the high turn count in this case reflects failure-driven retry behavior* rather than inherently longer successful task execution.

Agent Context Length. Figure 4 shows the accumulated context usage across all turns of a task, measured in thousands

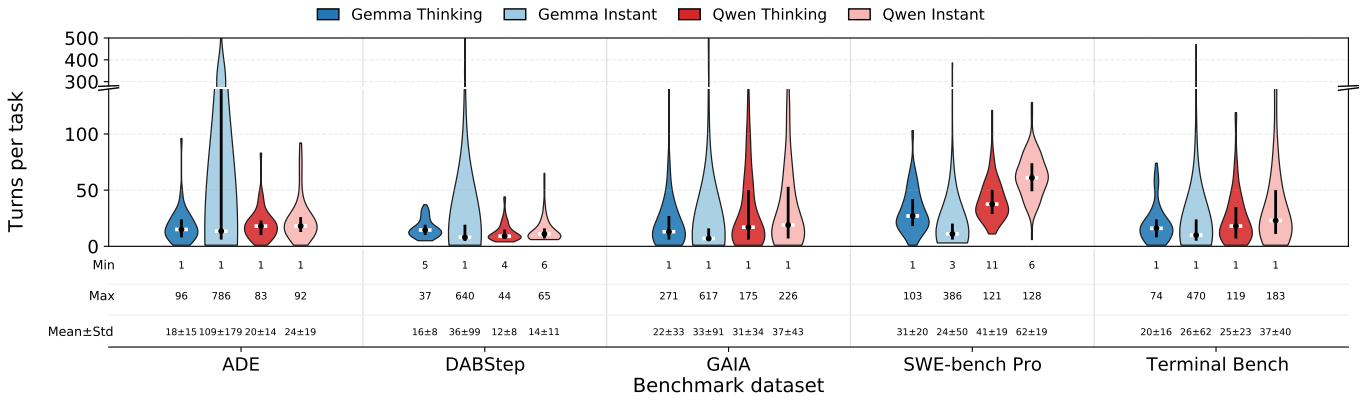

Fig. 3. Distribution of agent turns per task. Agent trajectories are highly variable: instant variants often require more turns and exhibit longer tails, while reasoning variants generally produce more compact interactions.

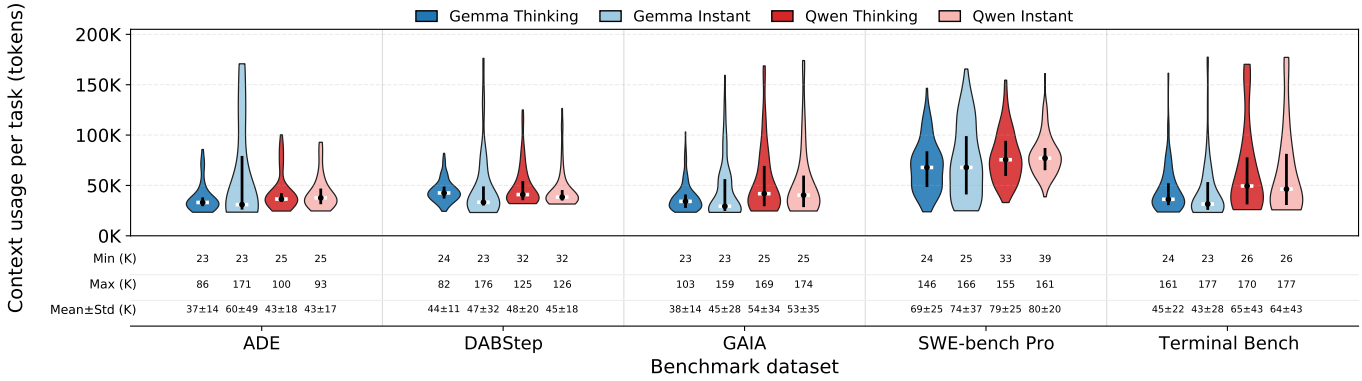

Fig. 4. Distribution of accumulated context usage across all turns for each task. Context usage varies substantially by workload, with SWE-bench Pro consistently producing the largest contexts, while instant variants often introduce longer tails even when their medians remain close to reasoning variants.

of tokens. Unlike turn count in Figure 3, context usage is less dominated by a few extreme trajectories and more consistently reflects workload complexity. *SWE-bench Pro is the most context-heavy benchmark* across all configurations, with mean context usage between 68.7K and 80.1K tokens and maxima between 146K and 166K tokens. Terminal Bench and GAIA also produce large contexts for Qwen, reaching mean usages of 63.5K–65.1K on Terminal Bench and 52.5K–54.5K on GAIA, while ADE and DABStep are smaller except for long-tail Gemma Instant cases.

The comparison between reasoning and instant variants shows that instant models do not always increase average context usage, but they often increase variance and tail behavior. Gemma Instant is especially variable: on ADE, it averages 60.3K tokens with a maximum of 171K, compared to 37.2K and 86K for Gemma Thinking; similar long tails appear on DABStep and GAIA. Qwen behaves differently: the thinking and instant variants have similar context usage on several datasets. However, Qwen is consistently more context-heavy than Gemma on GAIA and Terminal Bench.

Together with Figure 3, this suggests that *long agent trajectories and large accumulated contexts are related but not identical phenomena.* Some workloads require many model invocations because the agent proceeds through a long sequence of actions, while others accumulate large contexts because each turn carries substantial prompt, observation, tool-output, or intermediate reasoning state. This distinction is important because the two behaviors stress different parts of the serving stack: high turn counts increase scheduling frequency and repeated prefill/decode cycles, whereas large accumulated contexts increase KV-cache footprint, memory pressure, and context-reuse overheads. As a result, agentic serving systems must jointly optimize for repeated model re-entry and sustained context growth rather than treating sequence length or request count in isolation.

Agent Output Token Composition. Figure 5 shows how each model's generated output tokens are distributed across thinking tokens, normal message tokens, and tool-call tokens. For reasoning models, thinking tokens form a substantial fraction of generation: Gemma Thinking spends 45.8–67.6% of its output on thinking across workloads, while Qwen Thinking spends 29.0–40.7%. This confirms that enabling reasoning changes not only total token volume, but also the semantic composition of generated tokens. In contrast, instant variants

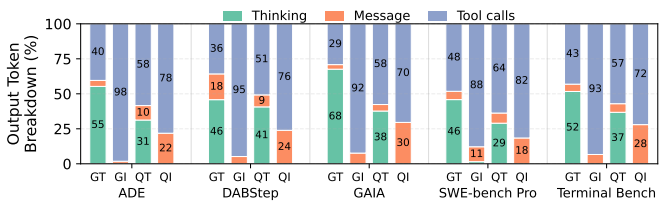

Fig. 5. Breakdown of generated output tokens into thinking, message, and tool-call tokens. Reasoning variants spend a large fraction of their output budget on thinking, while instant variants are dominated by tool-call generation.

generate no thinking tokens, so their output is split almost entirely between user-visible messages and tool calls.

An interesting result is that, the *tool-call tokens dominate output generation in many settings, especially for instant variants*. Gemma Instant spends 87.8–98.2% of its output tokens on tool calls, while Qwen Instant spends 70.4–81.6%. Even for reasoning variants, tool calls remain a large component: Qwen Thinking spends 50.8–63.7% of its output on tool calls in four of the five workloads, and Gemma Thinking reaches 48.2% on SWE-bench Pro. This shows that *ReAct-style agents are not simply producing long natural-language responses; much of their generated output is structured action generation that drives interaction with the environment.*

Successful Versus Failed Agent Context Lengths. Figure 6 compares accumulated context usage for successful and failed tasks. The overall trend is that failed agents usually accumulate larger contexts than successful agents, although the strength of this effect varies by workload and model. On GAIA, failures increase mean context usage for every configuration: from 34.6K to 40.6K tokens for Gemma Thinking, 36.4K to 47.3K for Gemma Instant, 45.6K to 64.7K for Qwen Thinking, and 44.9K to 62.4K for Qwen Instant. Terminal Bench shows an even sharper gap for Qwen, where failed runs reach 79.2K and 76.6K tokens on average for Qwen Thinking and Qwen Instant, compared to 44.4K and 45.9K for successful runs.

Failed agents often take more turns and also carry more accumulated context. The mechanism is consistent with Table IV: failed Bash, Edit, or Read actions append error messages, stack traces, failed patches, or diagnostic outputs into the agent history, which then become part of the next turn's input context. As a result, failed runs can amplify system load in two dimensions at once: they invoke the model more times and they increase the KV cache footprint carried across those invocations.

There are also useful exceptions. SWE-bench Pro is context-heavy for both successful and failed agents, and Gemma Instant has a higher mean context for successful runs than failed runs, 82.2K versus 72.3K tokens. This suggests that *large context is not inherently a failure mode*; some successful software-engineering tasks genuinely require sustained context about code, tests, and edits. The broader implication is that *outcome alone does not determine serving cost, but failed agents are a major source of long-tail context growth and therefore should be included explicitly when characterizing agentic serving workloads.*

Summary of Insights. *Agentic workloads are long-tailed in two distinct ways: some runs execute many turns, while others accumulate large contexts even with fewer turns. Surprisingly, reasoning does not uniformly increase workload cost; for Gemma, reasoning substantially reduces pathological long-turn trajectories, while instant variants often create longer tails. Failures also amplify system load: unsuccessful agents often execute more turns and carry larger contexts, meaning failed tasks can be expensive even when they do not produce useful outcomes.*

V. Runtime Performance Characterization

This section characterizes where agent execution time is spent and how context reuse shapes LLM serving cost. We ask four related questions: (1) how much time agents spend in LLM inference versus external tool execution, (2) how much of each turn's input context is reused through prefix/context caching versus newly appended, (3) whether LLM time is dominated by prefill or decode, and (4) how context-cache thrashing under memory pressure affects cache effectiveness and serving throughput.

LLM Versus Tool Call Time. Figure 7 decomposes total agent execution time into LLM inference and tool execution. Across all benchmarks and models, execution is primarily LLM-dominated, with LLM inference accounting for 71–98% of total runtime. DABStep is the most LLM-bound workload, spending 96–98% of time in the model, while ADE shows a similar trend with a larger but still secondary tool component.

However, tool execution is not negligible. Tools account for 2–29% of total runtime, with the largest fractions on GAIA, where tool time reaches 28.7% for Gemma Thinking and 24.9% for Qwen Thinking. This aligns with Table III, where GAIA includes expensive WebFetch, WebSearch, Agent, and TaskOutput calls. SWE-bench Pro and Terminal Bench also incur non-trivial tool costs, reaching 17.7% and 17.6%, respectively. Overall, these results show that *ReAct-style agent execution is dominated by repeated LLM invocations, but tool latency remains a substantial, workload-dependent component that serving systems should account for when scheduling, batching, or overlapping agent execution.* While the degree of concurrency may change the LLM v.s. tool time, our results are in line with other works [38].

Context Cache Effectiveness. Figure 8 decomposes the average tokens processed per agent turn into three components: input tokens that hit in the cache, newly appended input tokens that miss in the cache (i.e., cold miss), and generated output tokens. The dominant component is cached input context, which accounts for the vast majority of per-turn token volume in nearly every setting. The theoretical cache-hit ratio is consistently high, ranging from 87.9–99.3%, and the empirical cache-hit ratio is similarly high, ranging from 84.6–99.5%. This confirms that agentic execution contains

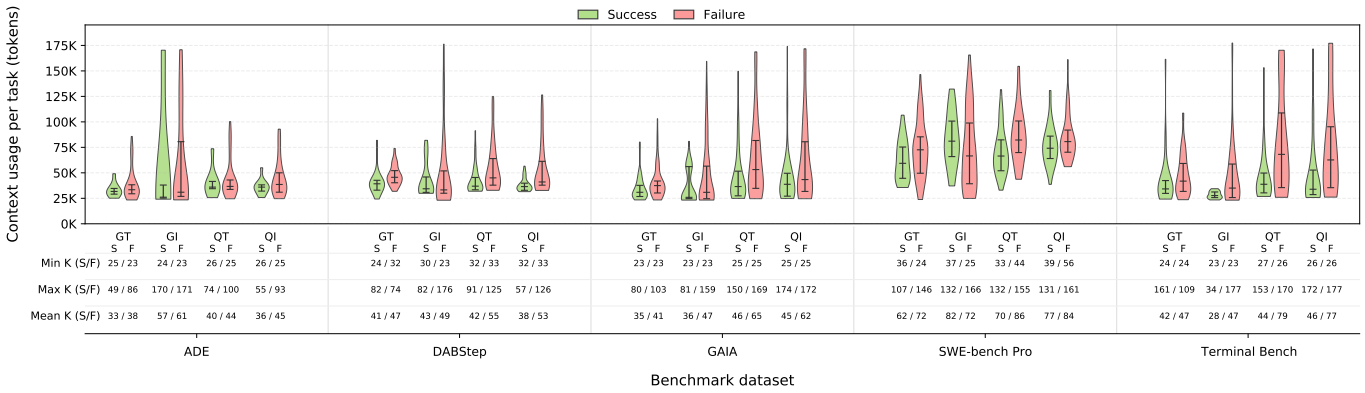

Fig. 6. Context-usage distributions split by task outcome. Failed agents often carry larger contexts (up to 1.8× in terms of mean) and longer tails, that runs can increase memory pressure even when they do not produce useful task outcomes.

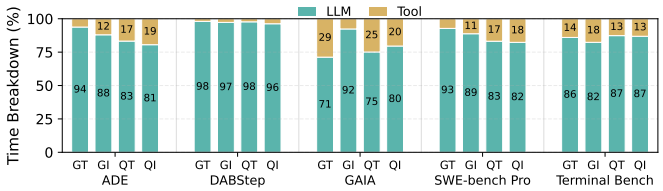

Fig. 7. Breakdown of end-to-end execution time between LLM inference and tool execution. Across workloads, most time is spent inside the LLM, but tool time becomes non-negligible for GAIA and Terminal Bench, reflecting stronger dependence on external interaction.

substantial prefix reuse: each turn reuses most of the accumulated history while only appending a relatively small amount of new information. Note that the measured hit rate is close to our estimation, and slight deviation is due to factors like system prompt sharing, inconsistency in tool call parser and chat template, etc.

Raw input-to-output ratios can mischaracterize agent workloads as overwhelmingly prefill-heavy, because the raw input includes accumulated history that is largely reused across turns. While the mean input-to-output ratio ranges from 53.9× to 559.8×, the mean append-to-output ratio is only 1.5×–7.3×, with medians often below 1.5×. Thus, the relevant per-turn prefill pressure is better captured by incremental appends rather than the full input sequence. Conceptually, this means *agent execution should not be modeled as a sequence of independent long-prompt requests, but as repeated model re-entry over a growing cached context.* Note that the cache hit rates we measure are close to the ideal rates expected in the absence of context-cache thrashing. With sufficient GPU memory, active contexts remain resident and cached tokens can be reused across turns; under memory pressure. However, thrashing repeatedly evicts useful cached prefixes from GPU memory, substantially lowering the effective hit rate and increasing prefill overhead.

Prefill Versus Decode Time. Figure 9 decomposes LLM execution time into prefill and decode phases. Across all benchmarks and models, LLM execution is strongly decode-dominated: decode accounts for 91.0–98.6% of LLM time, while prefill contributes only 1.4–9.0%. This is consistent with the high context reuse shown in Figure 8: although agents accumulate long contexts over many turns, most input tokens are reused from the cache, so each turn only prefills the appended region rather than the full history. We restate the same assumption here: this breakdown reflects a no-thrashing cache regime, where accumulated KV state over past conversations remains resident and is not evicted, recomputed, or fetched from slower memory tiers.

The largest prefill shares appear for Gemma Thinking on SWE-bench Pro and Gemma Instant on ADE, where prefill reaches 9.0% and 7.1%, respectively. Even in these cases, prefill remains a small fraction of LLM time because the append-to-output ratio is much lower than the raw input-to-output ratio: Figure 8 shows mean input-to-output ratios up to 559.8×, but mean append-to-output ratios of only 1.5×–7.3×. These results show that ReAct-style agent serving is better characterized as repeated decode over a growing cached context, rather than as repeated long-prompt prefilling.

Impact of Context Thrashing We quantify context thrashing by increasing concurrent agent jobs from Terminal-Bench and DABStep on a Qwen 3.6/H100 serving instance. The tables report prefix-cache hit rate, serving throughput, and request time spent in LLM prefill.

Context thrashing first appears as a sharp prefix-cache hit-rate drop, from about 98% to 16–26%, shifting the workload toward prefill. It has two effects: (a) extra prefill tokens slow decode for requests in generation, reducing average generation throughput by up to 66%; and (b) prefill time increases severalfold, though it remains below 10% of total request time. Thus, thrashing adds overhead through longer prefill and, more importantly, slower generation for inference batches. Our results assume recomputation under thrashing. Retrieving evicted KV states from secondary tiers, as in LMCache-style systems, is an alternative, but prior work shows such transfers can also be costly due to inter-tier bandwidth limits and retrieval latency [26].

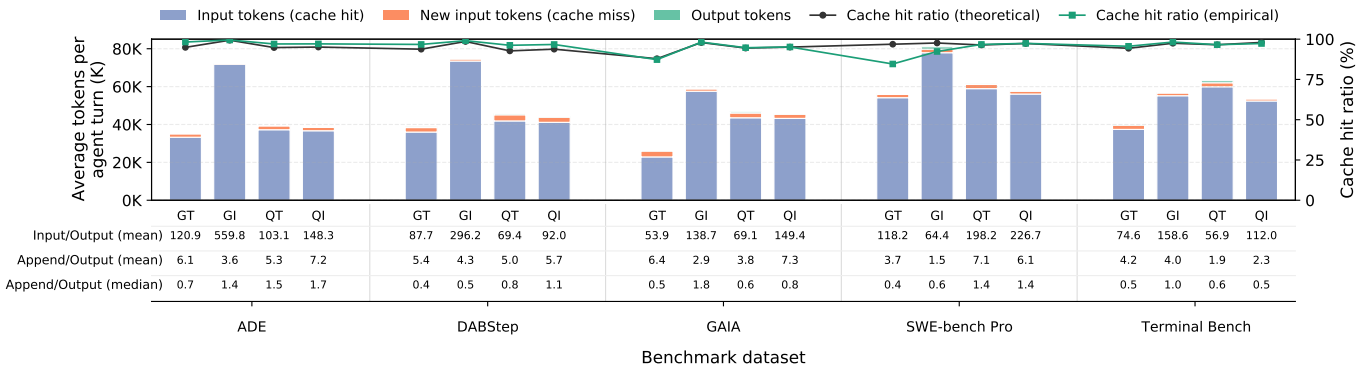

Fig. 8. Breakdown of per-turn context into cached input tokens, newly appended input tokens, and output tokens. Most input tokens are served from the prefix/context cache, yielding high cache-hit ratios, but each turn still appends a small stream of new tokens that must be processed and incorporated into the growing agent state. Here we investigate workload's intrinsic characteristic independent of whether it thrashes (i.e., context running out of GPU memory) in reality.

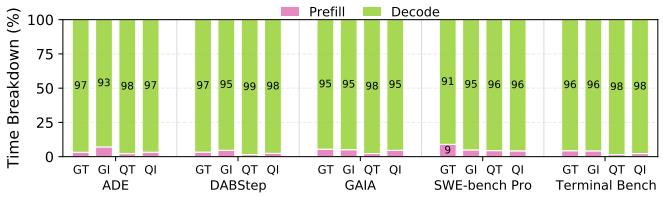

Fig. 9. Breakdown of LLM execution time into prefill and decode phases. Despite large accumulated contexts, decode dominates runtime because prefix/context caching turns most of the input into reusable state and leaves only a small newly appended region to prefill at each turn. We control the concurrency s.t. no context thrashing happens.

TABLE II
IMPACT OF CONTEXT THRASHING WITH INCREASING AGENT CONCURRENCY ON QWEN3.6-27B SERVING. † DENOTES CONFIGURATIONS WHERE CONTEXT THRASHING OCCURS. PREFILL TIME REPORTS BOTH ABSOLUTE TIME AND ITS FRACTION OF REQUEST TIME.

Terminal-Bench 2.0			
Max concurrency	15	20†	25†
Prefix-cache hit (%)	98.85	16.78	19.21
Gen. throughput (tok/s/req.)	30.08	16.96	10.30
Prefill time (s, %)	0.51 (4.42)	2.24 (8.38)	3.46 (7.81)
DABStep			
Max concurrency	5	10†	15†
Prefix-cache hit (%)	98.53	25.17	26.60
Gen. throughput (tok/s/req.)	34.13	22.21	14.82
Prefill time (s, %)	0.36 (5.07)	1.14 (6.57)	3.10 (7.27)

Summary of Insights. *Context caching can be a very effective technique to reduce the cost of prefill in agentic LLM serving. With ideal context caching, most input tokens are reused across turns, so the workload becomes dominated by the LLM decode phase rather than repeated full-context prefill. The implication is that efficient agent serving depends less on optimizing isolated long prompts and more on preserving long-lived KV/cache state across repeated model re-entry; losing that state would turn a decode-dominated workload into expensive recomputation.*

VI. Tool Call Characterization

This section characterizes how agents interact with external tools and how those interactions shape execution cost. We ask five related questions: (1) which tool types dominate agent behavior, (2) what concrete commands are issued through Bash, (3) which tools contribute the largest execution latency and returned observations, (4) which tools fail most often, and (5) how tool intent shifts over the course of an agent trajectory.

Breakdown by Tool Call Types. Figure 10 shows that tool-call composition varies much more by workload than by reasoning mode alone. ADE is dominated by repository-inspection and modification tools: Read accounts for 29.8–41.7% of tool calls in three configurations, while Bash contributes 23.2–30.0%. The main exception is Gemma Instant, where Edit alone accounts for 51.2% of all tool calls. DABStep and Terminal Bench are more execution-heavy. Bash accounts for 50.5–87.8% of tool calls on DABStep and 53.2–79.5% on Terminal Bench, indicating that these tasks rely heavily on command-line execution rather than a broad mix of specialized tools.

GAIA has a qualitatively different profile because web-facing tools become major components of agent behavior. WebFetch accounts for 42.3% of Gemma Thinking calls, 29.4% of Qwen Thinking calls, and 41.6% of Qwen Instant calls, while WebSearch reaches 29.7% for Gemma Instant. This explains why GAIA had the largest tool-time fraction in Figure 7: its tools are not merely frequent, but also include retrieval-oriented operations that can be slower and more variable than local file or shell operations. In contrast, SWE-bench Pro is more balanced across code-editing actions, with Read, Edit, Bash, and Grep all contributing substantially; for example, Gemma Thinking spends 29.9% of calls on Edit, 21.7% on Read, and 21.0% on Bash, while Qwen Instant spends 32.9% on Bash, 27.3% on Read, and 11.7% on Grep.

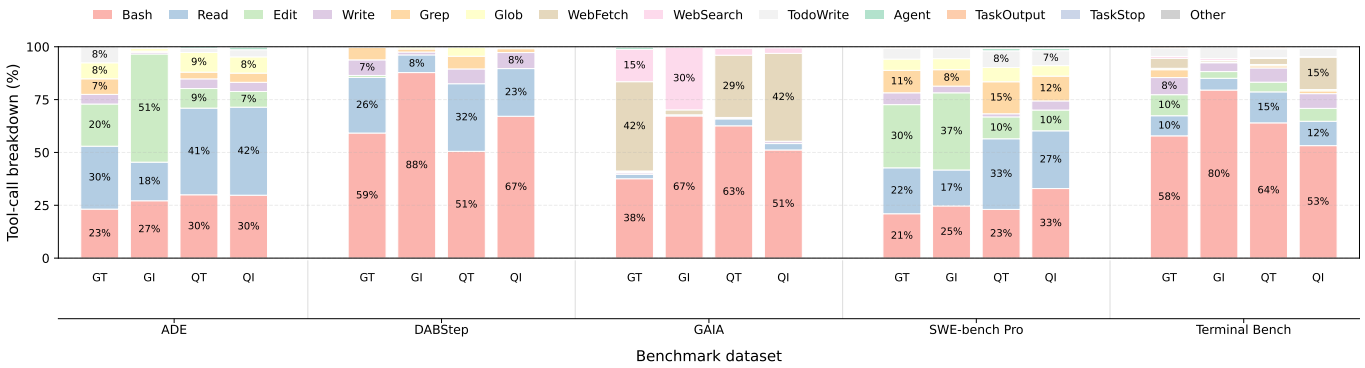

Fig. 10. Breakdown of tool-call types across workloads. Tool usage is highly model and domain-dependent: coding and terminal tasks are dominated by Bash, Read, and Edit operations, while GAIA introduces substantial WebFetch/WebSearch activity, reflecting its stronger dependence on external information retrieval.

The broader insight is that "tool use" is not a single systems category. A workload with many Bash calls stresses process execution and command-output handling, a workload with many Read/Edit/Grep calls stresses file-system interaction and codebase navigation, and a workload with WebFetch/WebSearch calls introduces external latency and retrieval variability. This distinction is important when interpreting earlier figures: similar tool-call counts per turn can correspond to very different runtime costs and context growth depending on which tools are invoked and how large their returned observations are.

Breakdown of Bash Command Types. Figure 11 breaks down Bash invocations by the concrete command executed. The key observation is that Bash is not a monolithic tool: the command mix changes substantially across workloads, revealing what the agent is actually doing after it decides to execute code or interact with the shell. ADE is dominated by dbt, which accounts for 24.7–50.9% of Bash commands across configurations, consistent with database-transformation style tasks. DABStep has a different structure: Gemma Instant uses grep for 54.7% of Bash commands, whereas Qwen relies heavily on python3, which accounts for 37.2–38.5% of its Bash commands. This suggests that even when agents operate on the same benchmark, different models may choose different execution strategies: one may inspect text through shell utilities, while another may script more of the task logic.

GAIA and the coding/terminal benchmarks expose another dimension of command diversity. GAIA includes a substantial share of curl commands, reaching 24.2–25.2% for Qwen and 24.8% for Gemma Instant, aligning with the earlier tool-type breakdown where web-facing tools were prominent. SWE-bench Pro and Terminal Bench have large Other fractions, reaching 50.0% for Gemma Thinking on SWE-bench Pro and 65.5% for Gemma Thinking on Terminal Bench. This category is not a single command; it aggregates locally rare commands, parsing artifacts such as 'parse_error' and 'empty', and global tail commands outside the top 12 shown in the legend. In practice, it includes meaningful shell, build, package, and workload-specific commands such as 'python', 'git', 'find', 'cat', 'pip', 'npm', 'node', 'make', 'gcc', 'g++', 'ffmpeg', 'pdflatex', and 'duckdb', so a large Other share indicates command diversity rather than simply noise.

The broader insight is that shell execution should be interpreted through the commands being run, not only through the number of Bash tool calls. A Bash call that runs grep, head, or cat is primarily an inspection operation, while one that runs python3, go, make, or benchmark-specific commands may execute tests, scripts, or workload logic and can produce very different latency and output-token footprints. This helps connect Figure 11 back to Figure 10: even when Bash dominates the tool mix, the underlying systems behavior can vary from file inspection to program execution, package/build activity, or data retrieval.

Top-3 Most Expensive Tool Calls. Table III summarizes the tools with the largest mean execution time for three representative workloads, while also marking tools that return large observations. Here, Agent refers to the delegation tool: a main-agent call that spawns a subagent to work on a bounded subproblem, rather than a direct shell, read, or edit action. Thus, when Agent appears as a top time-consuming tool, it indicates that a substantial portion of runtime is spent in delegated work. This is most visible in GAIA, where Agent takes 321.6s for Gemma-T, 916.0s for Qwen-T, and 650.7s for Qwen-I, suggesting that subagent-based reconnaissance, retrieval, or specialist analysis can dominate execution time.

The table also shows that expensive tools are often token-heavy. Many entries marked with † appear both among the slowest tools and among the largest result producers, meaning they affect serving twice: they stall agent progress and append large observations to the next-turn context. For example, on SWE-bench Pro, Agent returns 1,662–1,776 tokens for Qwen while taking roughly 58–59s per call, and TaskOutput returns 3,872 tokens for Gemma-I. Similarly, on ADE, Qwen's Agent calls return over 1.6K tokens while taking more than 56s per call. This connects directly to Figure 8: even when tool execution occurs outside the LLM, its returned observations become newly appended input tokens, increasing context growth, KV-cache footprint, and future model-side cost.

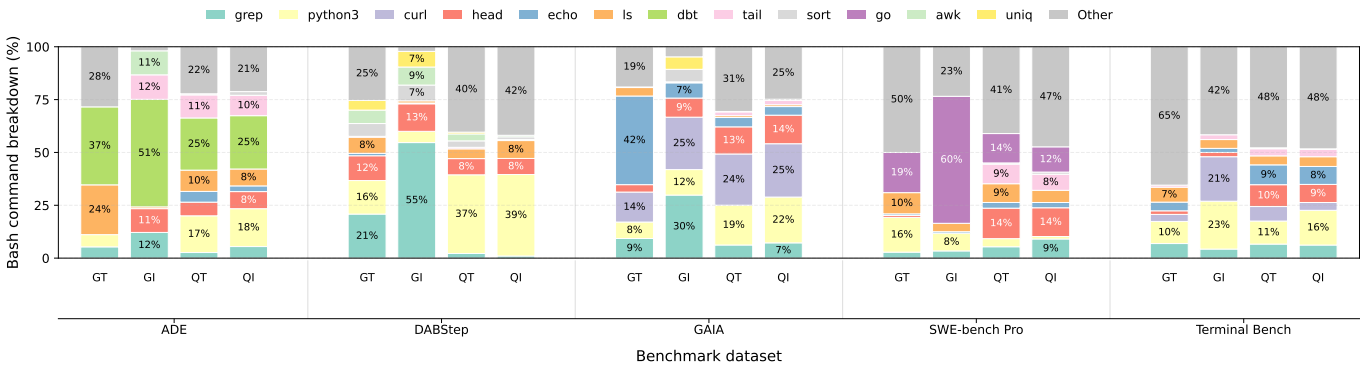

Fig. 11. Breakdown of Bash commands issued by agents. Command usage reflects the task domain: database-oriented workloads rely heavily on dbt, web/retrieval workloads use curl and inspection commands, and coding/terminal workloads use a broader mix of language runtimes, testing commands, and shell utilities.

Top-3 Failing Calls. Table IV reports tool types with lowest success rates for the three representative workloads. The main takeaway is that *failures concentrate in tools that modify state or execute code* (e.g., Bash and Edit). On ADE, Bash fails 28.4% for Gemma-T and 16.6–18.5% for Qwen, while Edit fails 95.4% for Gemma-I. SWE-bench Pro shows a similar pattern for Gemma, where Edit fails 37.2–77.8% and Bash fails 30.7–39.8%. These failures arise from actions that require precise interaction with the environment, such as patching files, running commands, or depending on repository state, dependencies, and test behavior.

GAIA exposes a different failure mode. Read is the top failing tool for Gemma, failing 48.9% for Gemma-T and 32.1% for Gemma-I, while web-related tools such as WebFetch fail at lower but non-trivial rates. Some 100.0% entries, such as AskUserQuestion, SendMessage, TaskStop, ExitPlanMode, and UpdateTodo, have only one or two calls and should therefore be interpreted cautiously. Compared with Table III, these results show that expensive tools are not always the most failure-prone: Agent is often latency-heavy, whereas Bash, Edit, and Read are more prominent sources of retries and error observations. This helps explain the long tails in Figures 3 and 4: failed tool calls can trigger additional diagnosis and retries, increasing both repeated model invocations and accumulated context.

Tool Intention Shift Over Time. Tool calls are made with intentions. We classfiy Read, Glob, Grep, WebFetch, WebSearch as tool calls with *Read/Explore* intention and Bash, Edit, Write, TodoWrite as the ones with *Execute/Write* intention. Figure 12 tracks how the fraction of tool calls devoted to these two intentions changes over the four quartiles of agent progress. A clear pattern emerges: *agents typically begin in an inspection-heavy mode and then transition toward action-heavy behavior.* This transition is strongest on DABStep, SWE-bench Pro, and Terminal Bench. For example, in several DABStep and SWE-bench Pro configurations, Read/Explore starts near 0.6–0.8 of tool calls in the first quartile of turns and falls close to zero by the final quartile, while Execute/Write rises from roughly 0.2–0.4 to 0.8–1.0. Terminal Bench shows a similarly strong action-heavy late phase, with some configurations such as Gemma Instant remaining execute/write dominated almost throughout.

ADE follows the same overall trend, though with more model variation. Gemma Thinking and both Qwen variants show a visible crossover around the midpoint of execution, where Read/Explore declines and Execute/Write becomes dominant; by the final quartile, execute/write often accounts for roughly 0.7–0.9 of tool calls. Gemma Instant on ADE is a notable exception in the other direction: it is already heavily execute/write oriented in the first quartile, suggesting that this configuration enters action mode earlier rather than spending many turns on broad exploration. While Terminal bench exhibits a less significant trend, it's because most of its tool calls are bash, which contains commands like 'ls' with exploration intention. By inspecting the specific bash commands, we do observe a more explicit trend. The broader insight is that *many agent trajectories appear to follow a natural pipeline: first inspect the environment or repository, then increasingly focus on acting on it.*

GAIA stands out as the main exception. Its curves are flatter and more balanced, with Read/Explore remaining substantial even late in execution and Execute/Write increasing only modestly. In configurations such as Gemma Thinking and Qwen Instant, the two fractions remain relatively close throughout the trajectory, ending near an approximate 40–60 split rather than converging to a strongly action-dominated phase. This is consistent with earlier results showing that GAIA is more retrieval-centric and tool-diverse: unlike software-engineering tasks, it often requires sustained information gathering instead of a clean transition into patching or test execution.

Summary of Insights. *Tool use is not a single systems category: the same "tool-call" abstraction can correspond to lightweight file inspection, expensive shell execution, web retrieval, or delegated subagent work. A surprising finding is that the tools that dominate latency are not always the tools that fail most often: 'Agent' and 'TaskOutput' can be latency- and token-heavy, while 'Bash', 'Edit', and 'Read' are more failure-prone and can trigger retries. Agents also exhibit a temporal structure,*

TABLE III

TOP TIME-CONSUMING TOOLS. EACH CELL REPORTS THE TOOL NAME, CALL COUNT n, MEAN DURATION μ, AND MEAN RESULT LENGTH r, WITH LINEARLY INTERPOLATED FIRST AND THIRD QUARTILES SHOWN AS $[Q_1, Q_3]$. † MARKS TOOLS THAT ARE ALSO AMONG THE TOP-5 BY RESULT LENGTH.

Benchmark	Rank	Gemma-T	Gemma-I	Qwen-T	Qwen-I
ADE	Top 1	**Bash**† n=208 μ=2.43s [0.09, 2.67] r=802 [32, 616]	**Bash**† n=1462 μ=2.56s [2.55, 2.71] r=480 [111, 513]	**Agent**† n=16 μ=66.31s [44.28, 82.19] r=1779 [1259, 2438]	**Agent**† n=35 μ=56.68s [32.31, 65.67] r=1626 [946, 2086]
	Top 2	**Grep**† n=66 μ=0.08s [0.07, 0.09] r=333 [33, 276]	**Write** n=41 μ=0.08s [0.05, 0.06] r=57 [44, 49]	**Bash**† n=611 μ=1.27s [0.10, 2.64] r=402 [64, 434]	**Bash**† n=747 μ=1.33s [0.11, 2.72] r=413 [58, 466]
	Top 3	**Write** n=41 μ=0.07s [0.05, 0.09] r=48 [41, 46]	**Edit**† n=2757 μ=0.07s [0.05, 0.08] r=123 [53, 129]	**TodoWrite** n=39 μ=0.10s [0.04, 0.06] r=40 [37, 37]	**WebSearch**† n=1 μ=0.15s [0.15, 0.15] r=233 [233, 233]
	Token-only extras	Read (r=718)	Glob (r=102)	Read (r=441)	Read (r=459), Glob (r=290)
GAIA	Top 1	**Agent**† n=51 μ=321.57s [107.08, 521.74] r=253 [176, 304]	**Agent**† n=6 μ=31.87s [24.83, 39.12] r=548 [291, 843]	**Agent**† n=11 μ=916.04s [207.90, 1262.39] r=420 [0, 671]	**Agent**† n=13 μ=650.73s [125.48, 929.74] r=650 [287, 781]
	Top 2	**WebFetch**† n=1810 μ=29.76s [9.59, 33.84] r=67 [29, 64]	**WebFetch**† n=123 μ=3.98s [0.79, 6.58] r=58 [33, 61]	**TaskOutput**† n=19 μ=298.45s [189.05, 323.00] r=2167 [126, 2758]	**TaskOutput**† n=23 μ=396.54s [302.89, 603.49] r=1260 [124, 1331]
	Top 3	**Bash**† n=1609 μ=2.06s [0.07, 0.30] r=410 [17, 123]	**Bash**† n=3551 μ=1.94s [0.12, 0.30] r=212 [17, 117]	**ScheduleWakeup** n=1 μ=20.67s [20.67, 20.67] r=41 [41, 41]	**Bash**† n=4631 μ=4.87s [0.21, 1.05] r=398 [19, 372]
	Token-only extras	–	–	Grep (r=1379), Read (r=620)	Read (r=675), Grep (r=258)
SWE-bench Pro	Top 1	**Agent**† n=3 μ=212.29s [35.20, 310.73] r=244 [118, 328]	**Agent**† n=8 μ=67.35s [34.89, 60.47] r=623 [573, 704]	**TaskOutput**† n=39 μ=90.65s [41.90, 120.98] r=459 [56, 472]	**TaskOutput**† n=49 μ=75.57s [30.68, 87.94] r=345 [57, 278]
	Top 2	**TaskOutput** n=22 μ=76.10s [40.77, 63.86] r=198 [65, 68]	**TaskOutput**† n=4 μ=20.02s [6.37, 33.51] r=3872 [1011, 4264]	**Agent**† n=75 μ=59.15s [42.43, 72.70] r=1662 [1259, 1881]	**Agent**† n=89 μ=58.47s [40.35, 71.26] r=1776 [1287, 2186]
	Top 3	**Bash**† n=691 μ=15.80s [0.09, 4.30] r=358 [26, 256]	**Bash** n=646 μ=10.68s [0.25, 2.49] r=243 [43, 309]	**Bash**† n=1473 μ=14.63s [0.08, 8.06] r=458 [58, 528]	**TaskStop** n=17 μ=8.88s [0.04, 2.00] r=120 [84, 95]
	Token-only extras	Grep (r=295), TaskStop (r=269)	EnterPlanMode (r=1157)	Read (r=2519), Grep (r=342)	Read (r=2232), Grep (r=246)

shifting from exploration early to execution/write behavior later; failed agents often make this shift too, but continue testing, debugging, or exhausting retrieval without reaching a correct terminal state.

VII. Limitations and Future Work

Our study focuses on a representative set of models and mainstream ReAct-style agent execution, leaving several directions for broader characterization in future work.

Larger Models. While larger models generally have stronger capabilities, some of the trends observed in our study still hold. For example, failed runs continue to consume more tokens. We evaluate GPT-5.5 on Terminal-Bench 2.0 and SWE-bench Pro, as shown in Table V. These results suggest that some workload trends persist even with larger models. Since our infrastructure will be open-sourced, these differences can be studied more extensively in future work.

Advanced Orchestration Techniques. Our study focuses on mainstream ReAct-style execution, including bounded one-level subagent delegation in Claude Code. Advanced orchestration techniques, including speculative or overlapped tool execution, dynamic context compaction or summarization, and branching or tree-search workflows, are outside the scope of this study. These techniques may change tool latency, context evolution, and KV-cache behavior, and we leave their characterization to future work.

VIII. Related Work

Prior work evaluates LLM agents through interactive benchmarks, progress metrics, framework efficiency, and infrastructure cost analysis [4], [5], [39], [40]. However, their focus is largely at the task, framework, or aggregate execution level, rather than on the fine-grained coupling between LLM serving and external tool execution. In contrast, we reconstruct end-to-end agent trajectories and correlate per-request serving traces with tool-level behavior to characterize how agentic workloads stress serving systems over time. Some other prior work spans foundational agent workflows such as ReAct, Reflexion, and LATS [3], [10], [12], infrastructure-cost characterization of dynamic reasoning [16], and recent

TABLE IV
TOP FAILING TOOL TYPES WITH EACH CELL REPORTING THE TOOL TYPE, CALL COUNT n, AND FAILURE RATE.

Benchmark	Rank	Gemma-T	Gemma-I	Qwen-T	Qwen-I
ADE	Top 1	**Bash** n=208, fail=28.4%	**Edit** n=2757, fail=95.4%	**Bash** n=611, fail=18.5%	**Bash** n=747, fail=16.6%
	Top 2	**Edit** n=179, fail=11.7%	**Bash** n=1462, fail=5.7%	**Edit** n=189, fail=16.4%	**Edit** n=186, fail=11.3%
	Top 3	**Read** n=267, fail=1.5%	**Glob** n=68, fail=1.5%	**Read** n=836, fail=1.8%	**Grep** n=109, fail=1.8%
GAIA	Top 1	**Read** n=90, fail=48.9%	**Read** n=28, fail=32.1%	**AskUserQuestion** n=1, fail=100.0%	**Bash** n=4631, fail=6.5%
	Top 2	**Bash** n=1609, fail=7.1%	**WebFetch** n=123, fail=7.3%	**SendMessage** n=1, fail=100.0%	**WebFetch** n=3761, fail=4.2%
	Top 3	**WebFetch** n=1810, fail=3.0%	**Bash** n=3551, fail=1.7%	**Read** n=226, fail=14.6%	**Write** n=85, fail=3.5%
SWE-bench Pro	Top 1	**TaskStop** n=1, fail=100.0%	**ExitPlanMode** n=1, fail=100.0%	**Bash** n=1473, fail=7.1%	**ExitPlanMode** n=2, fail=100.0%
	Top 2	**Edit** n=984, fail=37.2%	**Edit** n=955, fail=77.8%	**Edit** n=661, fail=1.5%	**UpdateTodo** n=1, fail=100.0%
	Top 3	**Bash** n=691, fail=30.7%	**Bash** n=646, fail=39.8%	**Write** n=94, fail=1.1%	**Bash** n=2820, fail=7.9%

Fig. 12. Progression of high-level tool intent over agent execution. Most agents shift from read/explore-heavy behavior early in execution to execute/write-heavy behavior later.

TABLE V
TOKEN USAGE OF SUCCESSFUL AND FAILED RUNS WITH GPT-5.5.

Model@Harness	Benchmark	Success	Fail
GPT-5.5@Terminus-2	Terminal-Bench 2.0	15,832.36	20,192.76 (+27.5%)
GPT-5.5@Codex	SWE-bench Pro	35,672.22	42,863.56 (+20.2%)

CPU/OS-centric studies of agentic execution [17], [41]. We focus on ReAct-style agents instantiated with Claude Code and correlate LLM-serving traces with tool-level behavior.

IX. CONCLUSION

This paper characterized ReAct-style agent workloads by connecting agent trajectories with LLM serving traces and tool-level behavior. The central finding was that *agentic serving was not simply long-prompt serving*: executions were long-tailed in both turn count and context growth, while effective context caching made the measured workload decode-dominated by reusing most prior context across turns. This *decode-bound behavior, however, represented an ideal cache regime*; in practical deployments, many concurrent agents and long multi-turn contexts can exceed GPU memory, forcing systems to choose between recomputation and offloading. We also found that *long-tail behavior could be amplified by failure-driven retry loops*, where repeated unsuccessful tool actions appended error observations and kept the agent interacting without meaningful progress. Finally, tool calls differed widely in latency, returned context, and failure behavior. Our characterization also exposed a clear temporal signal in agent behavior: *agents consistently move from exploration-heavy tool use early in execution toward action-oriented editing, testing, and finalization later.* Together, these results suggest that *efficient agentic serving requires jointly managing model execution, persistent context cache, tool interaction, and failure recovery rather than optimizing any one component in isolation.*

ACKNOWLEDGMENT

We used AI tools for code development, figure plotting, and text editing. This work was supported in part by the Semiconductor Research Corporation (SRC) under the GRC AIHW grant, the AMD AI & HPC Cluster Program, and Advanced Micro Devices, Inc. through the Funding Academic Research program.

References

[1] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko *et al.*, "Highly accurate protein structure prediction with alphafold," *Nature*, vol. 596, no. 7873, pp. 583–589, 2021.

[2] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of ai on developer productivity: Evidence from github copilot," *arXiv preprint arXiv:2302.06590*, 2023. [Online]. Available: https://arxiv.org/abs/2302.06590

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[4] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=zAdUB0aCTQ

[5] C. Ma, J. Zhang, Z. Zhu, C. Yang, Y. Yang, Y. Jin, Z. Lan, L. Kong, and J. He, "AgentBoard: An analytical evaluation board of multi-turn LLM agents," in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: https://openreview.net/forum?id=4S8agvKjle

[6] X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane, K. Sampath, M. Krishnan, S. Kundurthy, S. Hendryx, Z. Wang, V. Bharadwaj, J. Holm, R. Aluri, C. B. C. Zhang, N. Jacobson, B. Liu, and B. Kenstler, "SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks?" *arXiv preprint arXiv:2509.16941*, 2025.

[7] B. Stancil and dbt Labs, "ADE-bench: Analytics and data engineering benchmark," https://github.com/dbt-labs/ade-bench, 2026, accessed: 2026-05-21.

[8] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, "GAIA: A benchmark for general AI assistants," *arXiv preprint arXiv:2311.12983*, 2023.

[9] S. Yao, H. Chen, J. Yang, and K. Narasimhan, "Webshop: Towards scalable real-world web interaction with grounded language agents," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/82ad13ec01f9fe44c01cb91814fd7b8c-Abstract-Conference.html

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, 2023.

[11] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://openreview.net/forum?id=Yacmpz84TH

[12] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language agent tree search unifies reasoning, acting, and planning in language models," in *International Conference on Machine Learning*, 2024.

[13] Anthropic, "Claude Code: Overview," https://code.claude.com/docs/en/overview, 2026, accessed: 2026-05-21.

[14] M. Bolin, "Unrolling the Codex agent loop," https://openai.com/index/unrolling-the-codex-agent-loop/, Jan. 2026, openAI. Accessed: 2026-05-21.

[15] OpenClaw, "OpenClaw: Personal ai assistant," https://openclaw.ai/, 2026, accessed: 2026-05-22.

[16] J. Kim, B. Shin, J. Chung, and M. Rhu, "The cost of dynamic reasoning: Demystifying AI agents and test-time scaling from an AI infrastructure perspective," 2025. [Online]. Available: https://arxiv.org/abs/2506.04301

[17] R. Raj, S. Kundu, I. Vohra, H. Wang, and T. Krishna, "Towards understanding, analyzing, and optimizing agentic AI Execution: A CPU-centric perspective," 2025. [Online]. Available: https://arxiv.org/abs/2511.00739

[18] Harbor Framework Team, "Harbor: A Framework for Evaluating and Optimizing Agents and Models in Container Environments," https://github.com/harbor-framework/harbor, Jan. 2026, gitHub repository.

[19] Google DeepMind, "Gemma 4: Byte for byte, the most capable open models," https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/, Apr. 2026, accessed: 2026-05-21.

[20] Qwen Team, "Qwen3.6-27B: Flagship-level coding in a 27b dense model," https://qwen.ai/blog?id=qwen3.6-27b, Apr. 2026, accessed: 2026-05-21.

[21] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, 2023, pp. 611–626.

[22] A. Egg, M. Iglesias Goyanes, F. Kingma, A. Mora, L. von Werra, and T. Wolf, "Dabstep: Data agent benchmark for multi-step reasoning," *arXiv preprint arXiv:2506.23719*, 2025.

[23] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, J. Shen, G. Ye, H. Lin, J. Poulos, M. Wang, M. Nezhurina, J. Jitsev, D. Lu, O. Menis Mastromichalakis, Z. Xu, Z. Chen, Y. Liu, R. Zhang, L. L. Chen, A. Kashyap, J.-L. Uslu, J. Li, J. Wu, M. Yan, S. Bian, V. Sharma, K. Sun, S. Dillmann, A. Anand, A. Lanpouthakoun, B. Koopah, C. Hu, E. Guha, G. H. S. Dreiman, J. Zhu, K. Krauth, L. Zhong, N. Muennighoff, R. Amanfu, S. Tan, S. Pimpalgaonkar, T. Aggarwal, X. Lin, X. Lan, X. Zhao, Y. Liang, Y. Wang, Z. Wang, C. Zhou, D. Heineman, H. Liu, H. Trivedi, J. Yang, J. Lin, M. Shetty, M. Yang, N. Omi, N. Raoof, S. Li, T. Y. Zhuo, W. Lin, Y. Dai, Y. Wang, W. Chai, S. Zhou, D. Wahdany, Z. She, J. Hu, Z. Dong, Y. Zhu, S. Cui, A. Saiyed, A. Kolbeinsson, J. Hu, C. M. Rytting, R. Marten, Y. Wang, A. Dimakis, A. Konwinski, and L. Schmidt, "Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces," *arXiv preprint arXiv:2601.11868*, 2026.

[24] H. Li, Q. Mang, R. He, Q. Zhang, H. Mao, X. Chen, H. Zhou, A. Cheung, J. Gonzalez, and I. Stoica, "Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live," *arXiv preprint arXiv:2511.02230*, 2025.

[25] H. Kang, Z. Li, X. Yang, W. Xu, Y. Chen, J. Wang, B. Chen, T. Krishna, C. Xu, and S. Arora, "Thunderagent: A simple, fast and program-aware agentic inference system," *arXiv preprint arXiv:2602.13692*, 2026.

[26] Y. Cheng, Y. Liu, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, K. Du, and J. Jiang, "Lmcache: An efficient kv cache layer for enterprise-scale llm inference," *arXiv preprint arXiv:2510.09665*, 2025.

[27] Y. Yuan, M. Chowdhury, and N. Talati, "Kairos: Stateful, context-aware power-efficient agentic inference serving," 2026. [Online]. Available: https://arxiv.org/abs/2604.16682

[28] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, "Parrot: Efficient serving of LLM-based applications with semantic variable," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 929–945. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/lin-chaofan

[29] K. Santhanam, D. Raghavan, M. S. Rahman, T. Venkatesh, N. Kunjal, P. Thaker, P. Levis, and M. Zaharia, "Alto: An efficient network orchestrator for compound AI systems," in *Proceedings of the 4th Workshop on Machine Learning and Systems (EuroMLSys '24)*, 2024, pp. 117–125.

[30] Z. Asgar, M. Nguyen, and S. Katti, "Efficient and scalable agentic ai with heterogeneous systems," *arXiv preprint arXiv:2507.19635*, 2025.

[31] N. Wadlom, J. Shen, and Y. Lu, "Efficient llm serving for agentic workflows: A data systems perspective," *arXiv preprint arXiv:2603.16104*, 2026.

[32] OpenAI, "Introducing GPT-5.5," https://openai.com/index/introducing-gpt-5-5/, Apr. 2026, accessed: 2026-05-21.

[33] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: https://arxiv.org/abs/2405.15793

[34] vLLM Team, "Automatic Prefix Caching," https://docs.vllm.ai/en/latest/features/automatic_prefix_caching/, 2026, vLLM documentation. Accessed: 2026-05-21.

[35] Anthropic, "Claude Code: Create custom subagents," https://code.claude.com/docs/en/sub-agents, 2026, accessed: 2026-05-21.

[36] OpenTelemetry Authors, "OpenTelemetry Documentation," https://opentelemetry.io/docs/, 2026, accessed: 2026-05-21.

[37] Jaeger Authors, "Jaeger Documentation," https://www.jaegertracing.io/docs/latest/, 2026, accessed: 2026-05-21.

[38] A. Luo, "CPU-GPU Co-Design for Agentic LLM Inference," https://andyluo7.github.io/llm/amd/mi300x/vllm/lmcache/performance/2026/05/14/cpu-gpu-codesign-agentic-inference-mi300x/, May 2026, accessed: 2026-05-21.

[39] L. Gioacchini, G. Siracusano, D. Sanvito, K. Gashteovski, D. Friede, R. Bifulco, and C. Lawrence, "AgentQuest: A modular benchmark framework to measure progress and improve LLM agents," 2024. [Online]. Available: https://arxiv.org/abs/2404.06411
[40] Y. Xu, B. Zeng, Z. Qiu, Z. Zhang, G. Yue, X. Liao, H. Jin, and Q. Li, "AgentRace: Benchmarking efficiency in LLM agent frameworks," 2026, submitted to ICLR 2026. [Online]. Available: https://openreview.net/forum?id=eUuxWAQA5F
[41] Y. Zheng, J. Fan, Q. Fu, Y. Yang, W. Zhang, and A. Quinn, "AgentCgroup: Understanding and controlling OS resources of AI agents," 2026. [Online]. Available: https://arxiv.org/abs/2602.09345

Appendix

A. Abstract

This artifact accompanies our paper "Agentic AI Workload Characterization," which characterizes ReAct-style agents from both the LLM-serving and tool-execution perspectives. Our study covers two open-weight models, Gemma4-31B and Qwen3.6-27B, each in Thinking and Instant modes, across five agentic benchmarks.

The artifact contains both our trace collection/post-processing infrastructure and our collected data to support our paper. It provides detailed instructions for environment setup, data preparation, experiment execution, trace post-processing, and figure generation. We also provide our collected data for quick reproduction of the figures in our paper.

In particular, we focus on reproducing:

- A representative experiment using Qwen3.6-27B Thinking on ADE-Bench, including agent execution tracing and trace post-processing.
- Figures 2–11 in the main paper using our collected and post-processed data for all 20 benchmark/model configurations.

The first part validates our complete tracing and analysis pipeline. The second allows reviewers to confirm the paper's main results without re-collecting the data over several days on GPUs.

In addition to the above reproduction workflow for quick evaluation of the artifact, the artifact also provides complete instructions and scripts for collecting results for all 20 benchmark/model configurations.

B. Artifact check-list (meta-information)

- **Algorithm:** Trace correlation; Trace post-processing; workload analysis.
- **Program:** Python 3.13, Bash, Docker, Harbor, vLLM, and Jaeger (dependencies are managed by uv).
- **Compilation:** Not required.
- **Models:** Qwen3.6-27B and Gemma4-31B; Thinking and Instant modes.
- **Datasets:** ADE-Bench, DABStep, GAIA, SWE-bench Pro, and Terminal-Bench 2.0. Our raw and post-processed traces for all experiments in the paper.
- **Run-time environment:** Linux; uv; Docker; NVIDIA Container Toolkit.
- **Hardware:** CPU-only for reproducing the figures in the paper; two H100 NVL 80-GiB GPUs for the trace collection.
- **Run-time state:** Model weights and benchmark containers for trace collection.
- **Execution:** Example trace collection/post-processing pipeline; figure reproduction.
- **Metrics:** Turns; tokens; context reuse; LLM/tool and prefill/decode time; tool behavior.
- **Output:** agent execution traces; figures in the paper.
- **Experiments:** For quick evaluation: collect and post-process traces from Qwen3.6-27B Thinking running ADE-Bench; reproduce Figures 2–11. For full experiment: collect traces for all 20 configurations covered in the paper.
- **Disk space required:** Approximately 200 GB.
- **Time to prepare workflow:** Approximately 5–15 minutes after download.
- **Time to complete the experiments:** For quick evaluation: 30–60 minutes for the quick evaluation on Qwen 3.6/ADE-bench; a few minutes to reproduce the figures from our collected data. For full reproduction: 50–60 hours on two H100 GPUs for all 20 configurations.
- **Publicly available:** Yes.
- **Code license:** CC BY 4.0.
- **Data license:** CC BY 4.0.
- **Workflow automation:** Bash and Python scripts; uv; Docker.
- **DOI:** Code: https://doi.org/10.6084/m9.figshare.33188466.v1; data: https://doi.org/10.6084/m9.figshare.33188042.v1.

C. Description

1) How to access: The source code of the artifact can be accessed at https://figshare.com/articles/conference_contribution/IISWC_2026_AE_Source_Code/33188466.

We also provide our collected raw and post-processed data at https://figshare.com/articles/dataset/IISWC_2026_AE_Data/33188042.

The reviewer entry point is `README.ae.md` in the codebase, which includes an example agent trace collection/post-processing experiment and instructions for reproducing the figures in the paper.

To reproduce all data for all 20 configurations, readers should follow the instructions in the main `README.md`.

2) Hardware dependencies:

- We conducted our experiments on a server with two NVIDIA H100 NVL GPUs (80 GiB each), connected via NVLink.
- Agent trace collection uses two 80-GiB NVIDIA H100 NVL GPUs with tensor parallelism of 2. Figure reproduction using our collected data requires no GPU.

3) Software dependencies:

- Linux and Python 3.13.
- `uv`, `curl`, `tar`, `md5sum`, and `jq`. All Python dependencies are locked in `uv.lock`.
- Docker and NVIDIA Container Toolkit for fresh trace collection.
- vLLM 0.20.0 with the artifact's OTEL additions and Jaeger 1.57.

4) Datasets: The quick evaluation uses ADE-Bench as the agent task.

To reproduce the figures, we provide the data we collected for all 20 combinations of the five datasets and four model modes.

ADE-Bench/DABStep/GAIA/SWE-bench/Terminal-Bench-2.0 are used to reproduce the results of all experiments covered in this paper.

5) Models: Qwen/Qwen3.6-27B is used for the quick evaluation.

Qwen/Qwen3.6-27B and google/gemma-4-31B-it are used for the full reproduction of all results.

D. Installation

Download and extract `iiswc-2026-ae-main.zip`, enter the source root, and follow `README.ae.md`. Run `uv sync --frozen` to install the exact environment. Download `iiswc-2026-data-checkpoint.tar.gz` from the data DOI and follow `README.data.md` to verify and extract it.

E. Experiment Workflow

1) Download the source archive and data checkpoint. Follow `README.ae.md` for environment setup and `README.data.md` to verify and extract the checkpoint.
2) Reproduce Figures 2–11 by following `instructions/artifact-evaluation/iiswc-plots/README.md`.
3) To validate agent trace collection and post-processing infra with Qwen3.6-27B Thinking on ADE-Bench, follow `instructions/artifact-evaluation/data-collection-curation/README.md`.

F. Evaluation and expected results

For figure reproduction, we expect ten PDFs for Figures 2–11 that match the figures in the paper, together with ten summary CSV files. Figure 1 is a conceptual diagram and is not generated by the artifact.

For fresh data collection and processing, we expect new Qwen3.6-27B Thinking/ADE-Bench raw and processed data organized like the provided data. The exact numbers may differ due to the stochastic nature of LLM inference.